

Power Flow Solver Study Plan

Load Flow Lite

May 3, 2026

Contents

1 Purpose	2
2 Learning Outcomes	2
3 How to Use This Plan	2
4 Function Map	3
5 Glossary	3
6 Session 1: Get the Whole Solver in Your Head	4
7 Session 2: Understand Bus Types and the State Vector	5
8 Session 3: Build Ybus	6
9 Session 4: Convert Voltages Into Power Injections	6
10 Session 5: Build the Mismatch Vector	7
11 Session 6: Spend the Most Time on the Jacobian	8
12 Session 7: Work One Newton Step by Hand	9
13 Session 8: Interpret the Solved Result	10
14 Session 9: Validate the Complete Implementation	11
15 Common Confusions	11
16 Optional Theory Backup	12
17 Files to Delay Until Later	12
18 Final Self-Test	12

1 Purpose

This is a guided plan for understanding how this repository actually solves the AC power flow problem. Treat it as a lab, not a checklist: read a small amount of code, predict what it should do, run a focused test, then explain the result back in your own words.

The shortest path through the solver is:

1. Build the network admittance matrix, **Ybus**.
2. Convert a voltage guess into calculated bus injections.
3. Compare calculated injections with specified injections.
4. Assemble the Newton–Raphson Jacobian.
5. Solve a linear correction system.
6. Update voltage angles and magnitudes.
7. Repeat until mismatch is below tolerance.
8. Compute branch flows from the solved voltages.

2 Learning Outcomes

By the end, you should be able to:

- Explain which variables are solved for at slack, PV, and PQ buses.
- Trace how a **Case** becomes a **Ybus** matrix.
- Derive $S = V \cdot \text{conj}(Y_{\text{bus}}V)$ from voltage and current.
- Build the mismatch vector in the same order as the code.
- Map every block of the Jacobian to its rows, columns, and bus types.
- Hand-compute one Newton step for a two-bus slack/PQ system.
- Explain how solved bus voltages become branch powers and losses.
- Identify which files are core numerical code and which are wrappers.

3 How to Use This Plan

Make two passes.

Pass 1 is top-down. Start with `solve_power_flow` so you know the whole control flow before studying formulas.

Pass 2 is bottom-up. Study `build_ybus`, `power_injections`, `_mismatch`, and `_jacobian` in detail, then return to `solve_power_flow`.

For each session:

1. Read the listed functions.

2. Answer the checkpoint questions without looking ahead.
3. Run the focused test command.
4. Write down one sentence explaining what the code is doing mathematically.

The commands below assume the project virtual environment from the README:

```
python3 -m venv .venv
./venv/bin/pip install numpy pytest
```

4 Function Map

Use this as the map from concepts to code.

Concept	Main code	Why it matters
Case validation	<code>validate_case</code>	Rejects invalid topology before Newton starts.
Bus indexing	<code>bus_index</code>	Converts external bus IDs to array indices.
Network model	<code>build_ybus</code>	Builds the admittance matrix used in every injection calculation.
Complex voltages	<code>complex_voltages</code>	Converts magnitudes and angles into complex phasors.
Calculated injections	<code>power_injections</code>	Computes <code>P_calc</code> and <code>Q_calc</code> from the current voltage guess.
Specified injections	<code>specified_injections</code>	Applies the repo sign convention: generation minus load.
Bus type handling	<code>_bus_type_indices</code>	Determines which equations and unknowns exist.
Initial guess	<code>_initial_voltage</code>	Creates the first Newton state.
Mismatch vector	<code>_mismatch</code>	Defines the residual that Newton drives to zero.
Jacobian	<code>_jacobian</code>	Linearizes the AC power-flow equations.
Newton loop	<code>solve_power_flow</code>	Orchestrates the full nonlinear solve.
Result assembly	<code>_build_result</code>	Converts the converged state into useful outputs.
Branch flows	<code>compute_branch_flows</code>	Computes line powers and losses after convergence.

5 Glossary

Ybus. The bus admittance matrix. It maps complex bus voltages to complex injected currents: $\mathbf{I} = \mathbf{Ybus} @ \mathbf{V}$.

V. Complex bus voltage phasor. In code it is built from `v_magnitude` and `v_angle`.

theta. Voltage angle in radians.

P, Q. Real and reactive power injection. Positive means net injection into the network; negative means net load.

S. Complex power, $S = P + jQ$.

G, B. Real and imaginary parts of Y_{bus} , so $Y_{\text{bus}} = G + jB$.

Slack bus. Fixed voltage magnitude and fixed angle. Its real and reactive generation are solved outputs.

PV bus. Fixed real-power injection and voltage magnitude. Its angle is solved, and its reactive generation is a solved output.

PQ bus. Fixed real and reactive injection. Its voltage angle and magnitude are solved.

Mismatch. Specified injection minus calculated injection.

Jacobian. The matrix of partial derivatives used to turn a nonlinear mismatch into a linear correction step.

Per unit. Normalized power-system units. The repo stores powers in p.u.; conversion to MW/MVAr uses `base_mva`.

6 Session 1: Get the Whole Solver in Your Head

Read:

- `solve_power_flow`
- `_build_result`

Focus on the loop shape:

```
build Ybus
choose unknowns
initialize voltage state
repeat:
    compute mismatch
    stop if mismatch is small
    build Jacobian
    solve J * step = mismatch
    update angles and magnitudes
```

Checkpoint questions:

- Where is `Ybus` built?
- Where does the code stop on convergence?

- What arrays are updated during each Newton step?
- Why does non-convergence raise instead of returning a partial result?

Run:

```
./venv/bin/python -m pytest \
tests/test_solver.py::test_two_bus_power_flow_converges_and_balances_power -q
```

Done criteria:

- You can describe `solve_power_flow` without mentioning any wrapper code.
- You can point to the exact lines that call `_mismatch`, `_jacobian`, and `np.linalg.solve`.

7 Session 2: Understand Bus Types and the State Vector

Read:

- `_bus_type_indices`
- `_initial_voltage`
- `validate_case`

The Newton state is not all bus voltages. It is:

- Voltage angles for PV and PQ buses.
- Voltage magnitudes for PQ buses only.

The slack bus angle and magnitude stay fixed. PV bus voltage magnitudes also stay fixed, while their reactive generation is solved after convergence.

Checkpoint questions:

- What would go wrong if the slack angle were included as an unknown?
- Why does a PV bus solve for angle but not voltage magnitude?
- Why does a PQ bus solve for both angle and magnitude?
- For a system with 1 slack, 2 PV buses, and 6 PQ buses, how many Newton unknowns are there?

Run:

```
./venv/bin/python -m pytest \
tests/test_solver.py::test_pv_bus_keeps_voltage_magnitude_fixed -q
```

Done criteria:

- You can construct the state vector for any mix of slack, PV, and PQ buses.
- You can explain why PV-bus `q_gen` is not a fixed input to the mismatch.

8 Session 3: Build Ybus

Read:

- `build_ybus`
- `_branch_admittance_terms`

`Ybus` is the electrical network model. Each branch contributes self-admittance terms to the diagonal and mutual admittance terms to the off-diagonal entries. Line charging, transformer tap ratios, phase shifts, and bus shunts all enter here.

Checkpoint questions:

- Why does a simple series line add $+y$ to both diagonals and $-y$ to both off-diagonals?
- Why is line charging split between the two ends?
- What changes when `tap_ratio != 1.0`?
- Why can a phase shifter make `Ybus[from, to]` differ from `Ybus[to, from]`?

Run:

```
./venv/bin/python -m pytest tests/test_ybus.py -q
```

Done criteria:

- You can manually stamp a two-bus branch into a 2x2 `Ybus`.
- You can explain why `build_ybus` and `_branch_admittance_terms` must use the same branch model.

9 Session 4: Convert Voltages Into Power Injections

Read:

- `complex_voltages`
- `power_injections`

The key formula is:

$$\begin{aligned} I &= Y_{\text{bus}} V, \\ S &= V \cdot \text{conj}(I), \\ P_{\text{calc}} &= \text{real}(S), \\ Q_{\text{calc}} &= \text{imag}(S). \end{aligned}$$

This is where the current voltage guess becomes calculated real and reactive power injection at each bus.

Checkpoint questions:

- What are the shapes of V , $Y_{bus} @ V$, and S ?
- Why is the current conjugated in the complex-power formula?
- What does a negative P_{calc} mean?
- What does a negative Q_{calc} mean?

Run:

```
./venv/bin/python -m pytest \
tests/test_solver.py -k power_injection -q
```

Done criteria:

- You can compute $S = V * conj(Y_{bus} @ V)$ for a two-bus example.
- You can explain why this function knows nothing about slack, PV, or PQ bus types.

10 Session 5: Build the Mismatch Vector

Read:

- `specified_injections`
- `_mismatch`

The sign convention is:

```
P_spec = p_gen - p_load
Q_spec = q_gen - q_load
mismatch = specified - calculated
```

The mismatch vector is ordered as:

1. Real-power mismatches for PV and PQ buses.
2. Reactive-power mismatches for PQ buses.

Checkpoint questions:

- Why is there no real-power mismatch equation for the slack bus?
- Why is there no reactive-power mismatch equation for PV buses?
- How does the mismatch ordering match the Newton update vector?
- What would change if mismatch were defined as calculated minus specified?

Run:

```
./venv/bin/python -m pytest \
tests/test_solver.py::test_two_bus_power_flow_converges_and_balances_power -q
```

Done criteria:

- You can write the mismatch vector shape for a given set of bus types.
- You can explain the sign of the update in `solve_power_flow`.

11 Session 6: Spend the Most Time on the Jacobian

Read:

- `_jacobian`

This is the densest numerical function in the repo. It assembles the analytic polar AC power-flow Jacobian:

$$\begin{bmatrix} \partial P_{\text{calc}}/\partial\theta & \partial P_{\text{calc}}/\partial V \\ \partial Q_{\text{calc}}/\partial\theta & \partial Q_{\text{calc}}/\partial V \end{bmatrix}$$

Rows match equations:

- P rows for PV and PQ buses.
- Q rows for PQ buses.

Columns match unknowns:

- Angle columns for PV and PQ buses.
- Voltage-magnitude columns for PQ buses.

The implementation uses derivatives of calculated injections. Since mismatch is specified minus calculated, the Newton correction is applied consistently by solving:

$$\begin{aligned} J \cdot \text{step} &= \text{mismatch}, \\ x &\leftarrow x + \text{step}. \end{aligned}$$

Checkpoint questions:

- Which nested loop fills `dP/dtheta`?
- Which nested loop fills `dP/dV`?
- Which nested loop fills `dQ/dtheta`?
- Which nested loop fills `dQ/dV`?
- Why do diagonal and off-diagonal formulas differ?
- How do `g = ybus.real` and `b = ybus.imag` enter the formulas?

Run:

```
./venv/bin/python -m pytest tests/test_solver.py -q
```

Done criteria:

- You can label each quadrant of the Jacobian in the code.
- For a 1 slack, 1 PV, 2 PQ system, you can state the Jacobian dimensions and row/column ordering.

12 Session 7: Work One Newton Step by Hand

Use the two-bus case from `test_two_bus_power_flow_converges_and_balances_power`:

bus 1: slack, $V = 1.0$, $\theta = 0$
bus 2: PQ load, $P_{\text{load}} = 0.4$, $Q_{\text{load}} = 0.2$
branch: $r = 0.02$, $x = 0.04$, $b = 0.0$
flat start: $V_1 = 1.0$ angle 0, $V_2 = 1.0$ angle 0

First compute the series admittance:

$$y = \frac{1}{0.02 + j0.04} = 10 - j20.$$

So the flat-start Y_{bus} is:

$$Y_{\text{bus}} = \begin{bmatrix} 10 - j20 & -10 + j20 \\ -10 + j20 & 10 - j20 \end{bmatrix}.$$

At flat voltage, $Y_{\text{bus}} @ V = 0$, so:

$$\begin{aligned} P_{\text{calc},2} &= 0, & Q_{\text{calc},2} &= 0, \\ P_{\text{spec},2} &= 0 - 0.4 = -0.4, & Q_{\text{spec},2} &= 0 - 0.2 = -0.2. \end{aligned}$$

Therefore:

$$\text{mismatch} = \begin{bmatrix} -0.4 \\ -0.2 \end{bmatrix}.$$

For bus 2 at the flat start:

$$G_{22} = 10, \quad B_{22} = -20, \quad V_2 = 1, \quad P_{\text{calc},2} = 0, \quad Q_{\text{calc},2} = 0.$$

The 2x2 Jacobian is:

$$\begin{aligned} \frac{\partial P}{\partial \theta} &= -Q_{\text{calc},2} - B_{22}V_2^2 = 20, \\ \frac{\partial P}{\partial V} &= P_{\text{calc},2}/V_2 + G_{22}V_2 = 10, \\ \frac{\partial Q}{\partial \theta} &= P_{\text{calc},2} - G_{22}V_2^2 = -10, \\ \frac{\partial Q}{\partial V} &= Q_{\text{calc},2}/V_2 - B_{22}V_2 = 20. \end{aligned}$$

$$J = \begin{bmatrix} 20 & 10 \\ -10 & 20 \end{bmatrix}.$$

Solve:

$$J \cdot \text{step} = \begin{bmatrix} -0.4 \\ -0.2 \end{bmatrix}, \quad \text{step} = \begin{bmatrix} -0.012 \\ -0.016 \end{bmatrix}.$$

So the first update is approximately:

$$\begin{aligned}\theta_2 &= 0.0 - 0.012 = -0.012 \text{ radians,} \\ V_2 &= 1.0 - 0.016 = 0.984 \text{ p.u..}\end{aligned}$$

Checkpoint questions:

- Why is the first angle update negative?
- Why does the first voltage-magnitude update reduce V_2 ?
- Which two entries of the state vector exist in this two-bus case?
- Which entries would disappear if bus 2 were PV instead of PQ?

Done criteria:

- You can reproduce the first Newton step without reading the code.
- You can point from each hand-computed quantity to the line that computes it.

13 Session 8: Interpret the Solved Result

Read:

- `_build_result`
- `compute_branch_flows`
- `bus_power_balance_residuals`

After convergence, the code computes solved generation and branch flows. This is post-solution interpretation, not part of Newton convergence.

Checkpoint questions:

- Why is slack-bus generation computed after the solve?
- Why is PV-bus reactive generation computed after the solve?
- How are branch from-end and to-end currents computed?
- Why are branch losses the sum of the two terminal complex powers?

Run:

```
./venv/bin/python -m pytest tests/test_flows.py -q
```

Done criteria:

- You can explain how solved bus voltages become branch P/Q flows.
- You can distinguish mismatch convergence from post-solve reporting.

14 Session 9: Validate the Complete Implementation

Run the focused tests first:

```
./venv/bin/python -m pytest \
  tests/test_ybus.py tests/test_solver.py tests/test_flows.py -q
```

Then run the broader suite:

```
./venv/bin/python -m pytest -q
```

The small tests are best for understanding individual equations. The IEEE tests are best for confirming that the complete implementation matches static MATPOWER fixtures when those fixtures are present.

Done criteria:

- You know which test protects each concept.
- You can explain why a solver can pass a two-bus test but still need IEEE case validation.

15 Common Confusions

- Slack-bus `p_gen` and `q_gen` in the input are not fixed mismatch equations; the solved slack injection is computed from the final network state.
- PV-bus `q_gen` is solved, not fixed. This repo does not implement reactive power limits or PV-to-PQ conversion.
- PQ-bus voltage magnitude is unknown, even if the input provides an initial value.
- The mismatch excludes slack equations because the slack bus absorbs the active-power residual and fixes the angle reference.
- The mismatch excludes PV reactive equations because PV voltage magnitude is fixed and reactive output is allowed to move.
- `power_injections` computes injections for every bus, but `_mismatch` selects only the equations that belong in the Newton solve.
- `build_ybus` builds a global network model; `compute_branch_flows` uses the same branch model locally for each line after convergence.
- The code stores angles in radians. Some fixtures and reports may display degrees.
- The code solves `J * step = mismatch` because the implementation uses the Jacobian of calculated injections together with a specified-minus-calculated mismatch convention.

16 Optional Theory Backup

If you want more context while reading the code, use:

- `docs/ieee9_power_flow_lecture_note.tex`, especially the Newton–Raphson appendix.
- `docs/powerflow_v0_prompt.md`, which describes the original implementation goals and constraints.

Use these as references only after you have tried to trace the code path. The fastest learning loop is still: code, tiny example, test, explanation.

17 Files to Delay Until Later

These files are useful, but they are not the best starting point for understanding the numerical solve:

- `src/powerflow/io.py`: case loading.
- `src/powerflow/case.py`: data structures and validation.
- `src/powerflow/contingency.py`: repeated solves for outage analysis.
- `scripts/run_case9_n1.py`: command-line report wrapper.

Read these after the Newton–Raphson path is clear.

18 Final Self-Test

You are done with the core study plan when you can answer these without looking at the code:

- What is the Newton state vector for a network with 1 slack, 2 PV, and 6 PQ buses?
- Why is the slack bus excluded from both P and Q mismatch equations?
- Why is a PV bus included in P mismatch but excluded from Q mismatch?
- What does `S = V * conj(Ybus @ V)` compute?
- How is `P_spec` different from `P_calc`?
- What are the four Jacobian blocks?
- Why does the code update only `v_angle[angle_buses]` and `v_magnitude[pq_buses]`?
- How do branch flows differ from bus injections?
- Which function would you inspect first if voltages fail to converge?
- Which function would you inspect first if voltages converge but branch losses look wrong?